



UNIVERSIDAD DE LAS FUERZAS ARMADAS- ESPE - MATRIZ

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



ASIGNATURA

Metodología de la investigación

TEMA

Caja Blanca

INTEGRANTES

Jelen lucio, Jaime Cabezas, Bryan Cevallos

NIVEL-PARALELO - NRC

Tercero A – NRC : 30693

FECHA DE ENTREGA

19/06/2026

SANGOLQUI – ECUADOR

Requisito Funcional N.º REQ001

Sistema de Autenticación y Acceso

CÓDIGO FUENTE PARA VALIDAR EL LOGIN

```
@app.post("/login")
```

```
async def login(request: Request, form_data: OAuth2PasswordRequestForm = Depends()):
```

```
    registro = intentos_login.get(form_data.username)
```

```
    if registro:
```

```
        if registro["bloqueado_hasta"] and registro["bloqueado_hasta"] > datetime.now():
```

```
            raise HTTPException(status_code=429, detail="Demasiados intentos fallidos. Cuenta  
bloqueada por 5 minutos.")
```

```
        if registro["bloqueado_hasta"] and registro["bloqueado_hasta"] <= datetime.now():
```

```
            intentos_login[form_data.username]["intentos"] = 0
```

```
            intentos_login[form_data.username]["bloqueado_hasta"] = None
```

```
            db_usuario = await request.app.state.db["usuarios"].find_one({"correo":  
form_data.username})
```

```
            if not db_usuario or not pwd_context.verify(form_data.password,  
db_usuario["hashed_password"]):
```

```
                entrada = intentos_login.setdefault(form_data.username, {"intentos": 0, "bloqueado_hasta":  
None})
```

```
                entrada["intentos"] += 1
```

```
                if entrada["intentos"] >= 3:
```

```
                    entrada["bloqueado_hasta"] = datetime.now() + timedelta(minutes=5)
```

```
                    raise HTTPException(status_code=401, detail="Credenciales incorrectas")
```

```
intentos_login.pop(form_data.username, None)
```

```
expiracion = datetime.now(timezone.utc) +
```

```
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

```
payload = {"sub": db_usuario["correo"], "rol": db_usuario["rol"], "exp": expiracion}
```

```
token = jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

```
return {"access_token": token, "token_type": "bearer"}
```

DIAGRAMA DE FLUJO (DF) valida el login

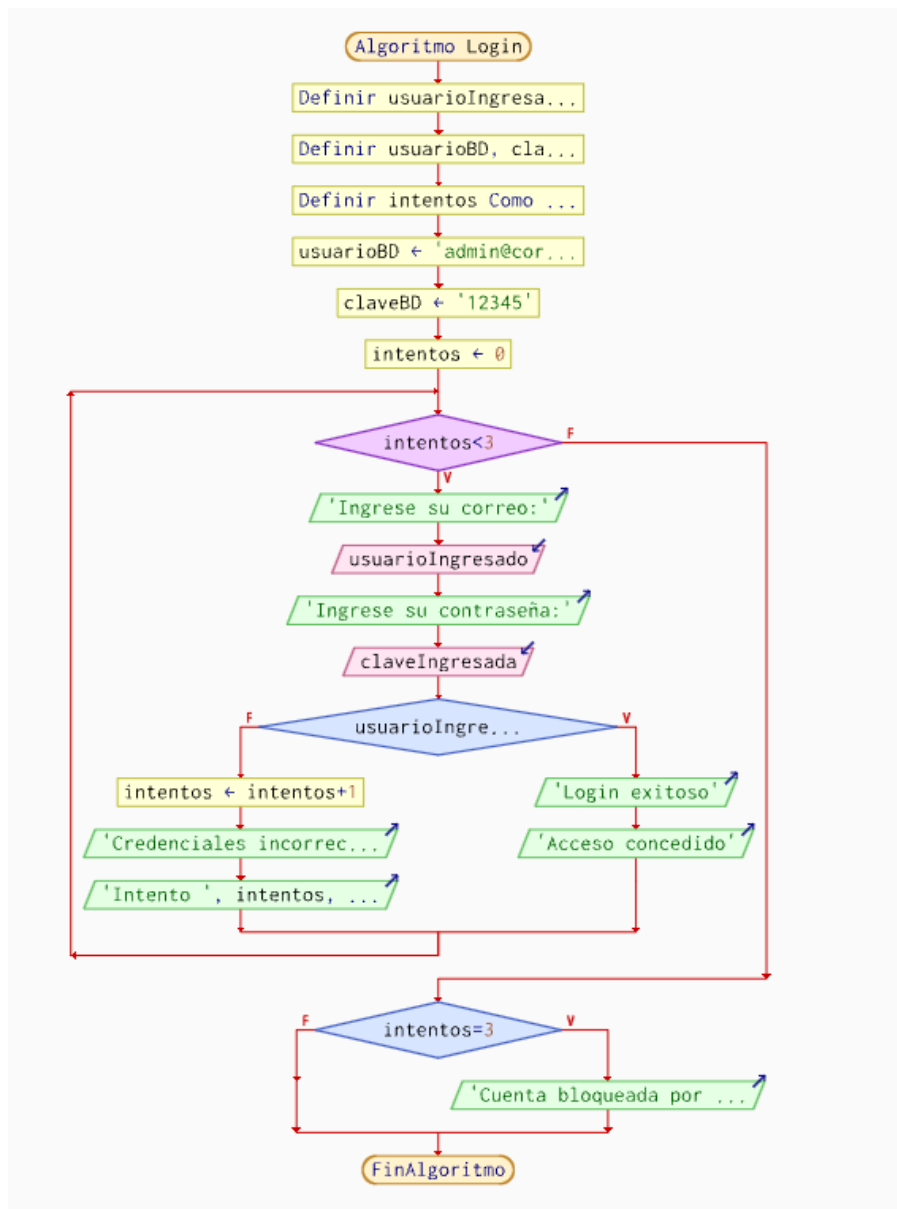
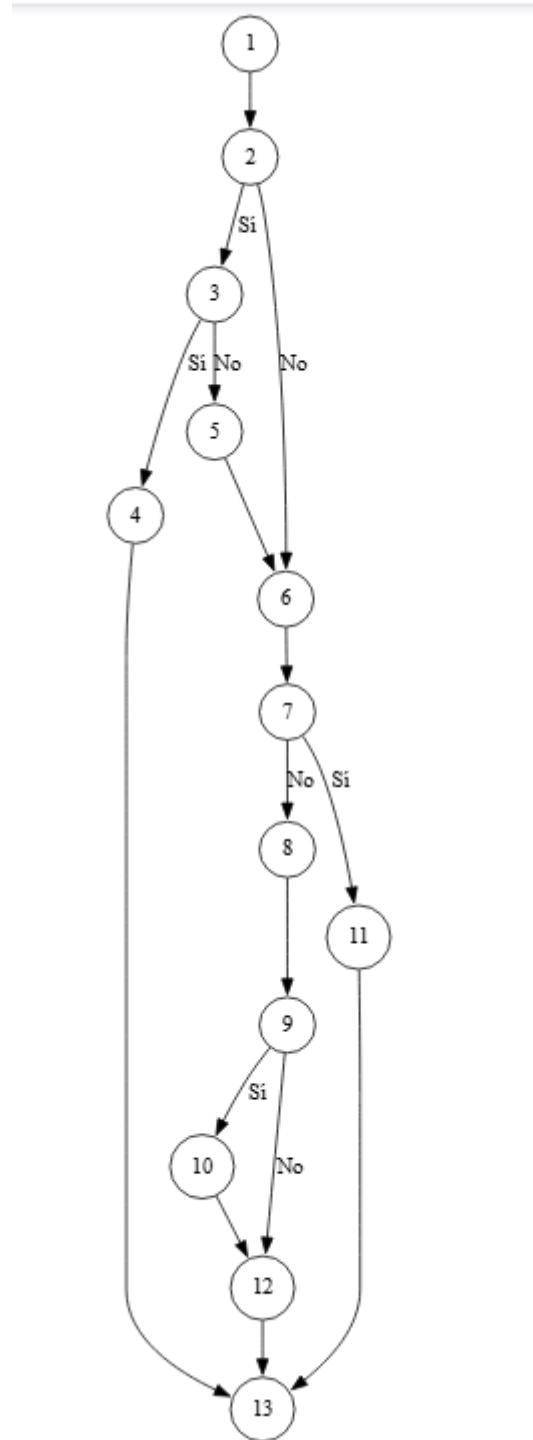


DIAGRAMA DE FLUJO (DF) valida el login



IDENTIFICACIÓN DE LAS RUTAS (Camino Básico) valida el login

Las rutas independientes identificadas en el Grafo de Flujo son:

R1: $1 \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow 11 \rightarrow 13$
(Login exitoso)

R2: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 13$
(Usuario bloqueado)

R3: $1 \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 12 \rightarrow 13$
(Credenciales incorrectas sin bloqueo)

R4: $1 \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 12 \rightarrow 13$
(Credenciales incorrectas con bloqueo por intentos)

COMPLEJIDAD CICLOMÁTICA valida el login

Método 1: Nodos predicado (IF-THEN-ELSE) + 1

Nodos predicado encontrados:

P1: ¿Existe registro?

P2: ¿Está bloqueado?

P3: ¿Bloqueo expirado?

P4: ¿Credenciales válidas?

P5: ¿Intentos ≥ 3 ?

Entonces:

$$V(G) = P + 1$$

$$V(G) = 5 + 1$$

$$V(G) = 6$$

Método 2: $V(G) = A - N + 2$

Donde:

N = 13 nodos

A = 17 aristas

Sustituyendo:

$$V(G) = A - N + 2$$

$$V(G) = 17 - 13 + 2$$

$$V(G) = 6$$

Resultado Final

$$V(G) = 6$$

Donde:

P = 5 (Número de nodos predicado)

A = 17 (Número de aristas)

N = 13 (Número de nodos)

Conclusión

La Complejidad Ciclomática del módulo de validación de login es 6, por lo que se requieren al menos 6 casos de prueba independientes para cubrir todos los caminos lógicos del código.